

VibeVolts Guide: Ground-Based Observatory Flow

VibeVolts Documentation

August 2, 2026

1 Introduction

This document traces the workflow for initializing, pointing, and recording observations using ground-based observatories in the VibeVolts simulation environment. It outlines how observatories are built, how they track targets, how imaging and radiometry are calculated, and how the data is recorded.

2 Building Observatories (Initialization)

Ground-based observatories are integrated directly into the global simulation state alongside space-based assets.

- **Allocation:** The function `add_observatories()` in `observatories.py` takes lists of latitudes, longitudes, and altitudes and updates `sim_data.observatories` with these locations.
- **Detector Integration:** Observatories share the same global detector array (`sim_data.detector`) as satellites. `add_observatories()` allocates blank detectors and appends them to this array.
- **Configuration:** The user configures these detectors (e.g., using `makeDetector()`), setting crucial parameters like Field of View (FOV), integration time, and the Earth exclusion angle (`earthEx`), which serves as the local horizon limit.

3 Pointing and Propagation

- **Propagation:** The `propagate_observatories()` function uses `astropy.coordinates.EarthLocation` to dynamically compute the Geocentric Celestial Reference System (GCRS) coordinates of the observatories as the Earth rotates.
- **Pointing:** Observatories are aimed by updating their respective vectors in `sim_data.detector.pointing`. In scenarios like `demo_observatory_scan.py`, this vector is computed directly as the difference between the target's position and the observatory's position.

4 Imaging and Radiometry

When the simulation advances (e.g., via `nextIntegration()`), the `scandetectors()` function in `scandetectors.py` performs the core imaging calculations:

- **Visibility & Exclusion:** The system calculates the angular separation between the detector pointing vector and the target vectors. The `exclusion()` function in `exclusion.py` ensures the target is not obscured by the Sun, Moon, or the Earth. For observatories, the Earth exclusion check enforces the local horizon limit defined by `earthEx`.
- **Flux Calculation:** For visible targets, `lambertiansphere()` computes the apparent brightness of the target (e.g., a satellite).
- **SNR Computation:** The signal is derived from the incident flux, integration time, aperture area, and quantum efficiency. The background noise (from space, sky, and sun) is factored in to yield the final Signal-to-Noise Ratio (SNR).

5 Recording Observations

The results from `scandetectors()` are returned as a dictionary containing simulation time, observer indices, target indices, signal, noise, and SNR.

- **DataHandler:** The `DataHandler` class in `dataHandling.py` is used to record this data across time steps.
- **Storage:** The `add_results()` method appends the step dictionary into a collection, which is subsequently consolidated into a Pandas `DataFrame` via `get_dataframe()`.
- **Export:** Finally, methods like `save_to_csv()` or `save_to_parquet()` can be used to export the complete observational dataset for analysis.